

Experiment No. – 8				
Date of Performance:	7/10/2025			
Date of Submission:	14/10/2025			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

Experiment No. 8

8.1 Aim: Implement PageRank using Map-Reduce.

8.2 Course Outcome: Adapt adequate perspectives of big data analytics in various applications like recommender systems, social media applications etc.

8.3 Learning Objectives: To enable students to have skills that will help them to solve complex real-world problems in decision support.

8.4 Requirement:

8.5 Related Theory:

PageRank is an algorithm used by search engines, such as Google, to rank web pages in their search results. It is a key component of Google's original search algorithm and is based on the idea that the importance of a web page can be determined by the number and quality of links pointing to it.

Map-Reduce is a programming model and processing framework commonly used for processing large datasets in a distributed computing environment, like Hadoop. It divides a task into two main phases: The Map phase, where data is divided into key-value pairs, and the Reduce phase, where these pairs are processed and aggregated. PageRank can be implemented using Map-Reduce to handle the immense amount of data on the web efficiently.

What is the damping factor?

According to the PageRank theory, a hypothetical surfer who clicks on links at random will soon stop clicking at some point in time. A damping factor d is the probability that the individual will continue to surf at any point. Different damping factors have been investigated in various experiments; however, it is generally anticipated that the damping factor will be fixed at around 0.85.

Here's a high-level overview of how you can implement PageRank using Map-Reduce:

1. Data Representation:

- Each web page is represented as a node in a graph.

- Links between web pages are represented as directed edges between nodes.
2. Initialization:
 - Assign an initial PageRank score to each node. Typically, you start with a uniform score for all nodes.
 3. Map Phase:
 - For each web page (node) in the dataset, emit its ID and its current PageRank score.
 - For each outgoing link from a web page, calculate and emit a contribution to the linked page's PageRank score. This contribution is the current page's PageRank divided by the number of outgoing links.
 4. Reduce Phase:
 - For each node, sum up all the contributions it receives from other nodes.
 - Apply the PageRank formula to update the PageRank score for each node. The formula is: $\text{newPageRank} = (1 - \text{damping factor}) / N + \text{damping factor} * (\text{sum of contributions})$, where the damping factor is a value between 0 and 1 that represents the probability of a user clicking on a random link, and N is the total number of nodes.
 5. Iteration:
 - Repeat the Map-Reduce process for a specified number of iterations or until convergence (when the PageRank scores stabilize).
 6. Sorting:
 - After the final iteration, you can sort the nodes by their PageRank scores to rank the web pages.
 7. Output:
 - The final output is a list of web pages ranked by their PageRank scores.

This iterative Map-Reduce process is repeated until the PageRank scores converge to a stable state, typically with decreasing changes between iterations. It's worth noting that a real-world implementation of PageRank using Map-Reduce also involves handling issues like dead ends (pages with no outgoing links) and dealing with the damping factor.

In summary, PageRank using Map-Reduce is a scalable and efficient way to calculate the importance of web pages in large-scale web graphs by breaking down the computation into smaller, parallelizable tasks.

Applications:

- The PageRank algorithm's earliest and most obvious application is in search engines. PageRank can rank websites to give more relevant search results faster because it was built expressly for use in Google's search engine.
- The PageRank algorithm is used to search networks that are not connected to the

internet. This can be used in academic papers; PageRank can determine the most effective and referenced publications in a field by utilizing citations instead of links.

- An example of a real-world application of the PageRank method is identifying significant species in an ecology. Applying the PageRank algorithm to the relationships between species in an ecosystem allows the user to determine the most notable species. As a result, assigning value to essential animal and plant species in an ecosystem makes it easier to predict outcomes such as extinction or a species' removal from the ecosystem.

8.6 Procedure:

Pseudocode:

```
map( key: [url, pagerank],
    value: outlink_list ) for each
    outlink in outlink_list
        emit( key: outlink, value: pagerank/size(outlink_list) )

emit( key: url, value: outlink_list )

reducer( key: url, value: list_pr_or_urls )
    outlink_list = []
    pagerank = 0

    for each pr_or_urls in
        list_pr_or_urls if
            is_list( pr_or_urls )
                outlink_list =
                    pr_or_urls else
                        pagerank += pr_or_urls

    pagerank = 1 - DAMPING_FACTOR + ( DAMPING_FACTOR *

    pagerank ) emit( key: [url, pagerank], value: outlink_list )
```

8.7 Program and Output:

```
data = """A\t1.0\tB,C,D
B\t1.0\tC
C\t1.0\tA
D\t1.0
"""

with open("pagerank_data.txt", "w") as f:
    f.write(data)

print("pagerank_data.txt created successfully!\n")
!cat pagerank_data.txt
```

pagerank_data.txt created successfully!

A	1.0	B,C,D
B	1.0	C
C	1.0	A
D	1.0	

```
%%writefile mapper.py
#!/usr/bin/env python3
import sys

for (variable) parts: list[str]
    parts = line.strip().split('\t')
    if len(parts) < 3:
        continue
    page = parts[0]
    rank = float(parts[1])
    outlinks = parts[2].split(',')

    if len(outlinks) > 0:
        contribution = rank / len(outlinks)
        for dest in outlinks:
            print(f"{dest}\t{contribution}")

    print(f"{page}\t{'|'.join(outlinks)}")
```

Writing mapper.py



```
%%writefile reducer.py
#!/usr/bin/env python3
import sys

damping = 0.85
current_page = None
rank_sum = 0.0
outlinks = []

for line in sys.stdin:
    parts = line.strip().split('\t')
    if len(parts) != 2:
        continue
    page, value = parts

    if current_page != page:
        if current_page:
            new_rank = (1 - damping) + damping * rank_sum
            outlinks_str = ','.join(outlinks)
            print(f"{current_page}\t{new_rank}\t{outlinks_str}")
        current_page = page
        rank_sum = 0.0
        outlinks = []

    if '|' in value:
        outlinks = value.split('|')
    else:
        try:
            rank_sum += float(value)
        except:
            pass

if current_page:
    new_rank = (1 - damping) + damping * rank_sum
    outlinks_str = ','.join(outlinks)
    print(f"{current_page}\t{new_rank}\t{outlinks_str}")
```

[How can I ir](#)

 !cat reducer.py

```
 #!/usr/bin/env python3
import sys

damping = 0.85
current_page = None
rank_sum = 0.0
outlinks = []

for line in sys.stdin:
    parts = line.strip().split('\t')
    if len(parts) != 2:
        continue
    page, value = parts

    if current_page != page:
        if current_page:
            new_rank = (1 - damping) + damping * rank_sum
            outlinks_str = ','.join(outlinks)
            print(f"{current_page}\t{new_rank}\t{outlinks_str}")
        current_page = page
        rank_sum = 0.0
        outlinks = []

    if '|' in value:
        outlinks = value.split('|')
    else:
        try:
            rank_sum += float(value)
        except:
            pass

if current_page:
    new_rank = (1 - damping) + damping * rank_sum
    outlinks_str = ','.join(outlinks)
    print(f"{current_page}\t{new_rank}\t{outlinks_str}")
```

```

import subprocess, os

input_file = "pagerank_data.txt"
iterations = 5 # You can increase this for more accuracy

for i in range(iterations):
    print(f"\n--- Iteration {i+1} ---")
    mapper = subprocess.Popen(['python3', 'mapper.py'], stdin=open(input_file), stdout=subprocess.PIPE)
    reducer = subprocess.Popen(['python3', 'reducer.py'], stdin=mapper.stdout, stdout=open('output.txt', 'w'))
    mapper.stdout.close()
    reducer.communicate()
    os.replace('output.txt', input_file)

print("\nFinal PageRank Results:\n")
!cat pagerank_data.txt

```

```

--- Iteration 1 ---

--- Iteration 2 ---

--- Iteration 3 ---

--- Iteration 4 ---

--- Iteration 5 ---

Final PageRank Results:

B      0.19250000000000003
C      0.19250000000000003
D      0.19250000000000003
A      0.15000000000000002      B,C,D

```

8.8 Conclusion:

The implementation of PageRank using MapReduce demonstrates an efficient and scalable approach to compute the importance of web pages in large-scale graphs. By distributing the computation of rank contributions across multiple nodes, MapReduce allows each page's rank to be updated in parallel, significantly reducing processing time for massive datasets. The mapper phase efficiently distributes rank values to linked pages, while the reducer phase aggregates these contributions and applies the damping factor to update page ranks. Iterating this process ensures convergence to stable PageRank values. Overall, this approach leverages the power of distributed computing to handle large web graphs, making PageRank practical for real-world applications in search engines and network analysis.

8.9 Questions:

4 to 5 MCQ/Fill in the Blanks

- A. Which mechanism in Hadoop is **most efficient** to detect PageRank convergence between two iterations?
 - a. Using Hadoop counters to track total rank change across reducers
 - b. Using custom input format that reads both old and new ranks
 - c. Using speculative execution to compare partial ranks
 - d. Using distributed cache to store rank deltas

- B.** In a MapReduce PageRank job, how should **dangling nodes** (pages with no outlinks) be treated within the Mapper code to avoid rank leakage?
- a. Emit nothing for dangling nodes
 - b. Emit a dummy self-loop edge (page, page)
 - c. Accumulate their rank contributions using a Hadoop counter and redistribute in the next iteration
 - d. Assign their rank to a default “sink” node
- C.** In a Hadoop-based PageRank implementation, each iteration’s **Mapper** emits both the adjacency list and rank. Contributions. Which code design pattern ensures that **the graph structure (outlinks)** is preserved after the Reduce phase?
- a. Emit adjacency list from Mapper only for pages with outlinks
 - b. Emit the adjacency list with a unique marker (e.g., “@”) so Reducer can distinguish structure from rank values
 - c. Store adjacency list in Distributed Cache
 - d. Skip adjacency list emission to reduce shuffle overhead
- D.** PageRank implemented with MapReduce often requires **multiple iterations**. Which factor has the **strongest effect** on the number of iterations needed for convergence?
- a. Size of input graph (N and M)
 - b. Damping factor value (α)
 - c. Number of reducers used
 - d. HDFS replication factor
- E.** In large-scale PageRank computation, the algorithm may encounter **rank sink problems**. Which MapReduce-level strategy best prevents this?
- a. Assigning zero rank to all dangling nodes
 - b. Normalizing rank values at the Reducer output
 - c. Redistributing rank mass from dangling nodes equally among all nodes per iteration
 - d. Using a single reducer for consistent aggregation